Structured Query Language (SQL 2)
Tercera Parte

3. *Ejercicios Globales*

Ejercicio 1:

Armar la tabla de pasaje del Álgebra Relacional al SQL.

Algebra Relacional	Structured Query Language (SQL)
$r \cup s$	
$r \times s$	
$r - s$	
$\pi_{A1, A2..Ak}(r)$	
$\sigma_{cond}(r)$	

Respuesta:

Algebra Relacional	Structured Query Language (SQL)
$r \cup s$	`(SELECT * FROM r ) UNION (SELECT * FROM s)`
$r \times s$	`SELECT DISTINCT * FROM r, s`
$r - s$	`(SELECT * FROM r) EXCEPT (SELECT * FROM s)`
$\pi_{A1, A2..Ak}(r)$	`SELECT DISTINCT A1, A2, ....Ak FROM r`
$\sigma_{cond}(r)$	`SELECT DISTINCT * FROM r WHERE cond'`

Ejercicio 2:

Sea el siguiente esquema de base de datos, suponiendo que las relaciones contienen las siguientes tuplas:

DEPARTAMENTO	
nombre	**jefe**
RRHH	Ana
Tesoreria	Jose
Ventas	Sergio
Hardware	Noemi
Software	Hernan

EMPLEADO		
nombre	**sueldo**	**trabaja_en**
Gladys	1300	RRHH
Jose	1850	Tesoreria
Claudio	1930	Ventas
Sergio	2000	Ventas
Pedro	2200	Hardware
Pablo	1900	Hardware
Noemi	2500	Hardware
Diego	3500	Null
Ana	1700	RRHH
Juan	1600	RRHH
Clara	1800	Software
Hernan	1950	Software

Silvia Alicia Gómez & Leticia Irene Gómez

PROVEEDOR	
nombre	**direccion**
Compucompras	Cordoba 1200
CDMarket	Rivadavia 6585
Florida I	Florida 50
Compushow	Florida 537
IBM	Catalinas 1100

ARTICULO			
codigo	**descripcion**	**stock**	**vendido_por**
10	visual age for smalltalk	5	Software
20	visual age for c++	3	Software
30	visual age for cobol	1	Software
40	db2	7	Software
60	pentium II	3	Hardware
70	compaq presario	2	Hardware
100	ibm 3709 notebook	2	Hardware
550	monitor de cristal liquido	0	Hardware

PROVEE		
codigo	**nombre**	**precio**
10	IBM	350
20	IBM	370
30	IBM	270
40	IBM	1100
60	Compucompras	1100
60	CDMarket	1099
60	Florida I	1200
100	IBM	2500
100	Compucompras	2650
100	CDMarket	2550
100	Florida I	2700
100	Compushow	3200
70	Florida I	5200

Ejercicio 2.1)

Obtener las direcciones de los proveedores que estén en la calle Florida. La nueva relación deberá tener como nombre de atributo DIR. En el ejemplo se debería obtener:

DIR
Florida 50
Florida 537

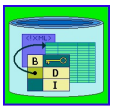
Respuesta 2.1)

```
SELECT  direccion AS dir
FROM proveedor
WHERE direccion LIKE 'Florida%'
```

Muy importante

Existen DBMS que guardan los datos case-sensitive. Generalmente provienen del mundo Unix como por ejemplo Oracle. Si las tuplas se hubieran insertado con “florida” en minúsculas jamás se hubiera recuperado el resultado.

Silvia Alicia Gómez & Leticia Irene Gómez



Ejercicio 2.2)

Listar los nombres de empleados que tienen por lo menos seis caracteres. En el ejemplo se debería obtener:

Nombre
Gladys
Claudio
Sergio
Hernan

Respuesta 2.2)

```
SELECT  nombre
FROM empleado
WHERE nombre LIKE '______%'
```

Ejercicio 2.3)

Hallar las descripciones de los artículos junto con los nombres de los proveedores que los proveen. En el ejemplo debería obtenerse:

descripcion	nombre
visual age for smalltalk	IBM
visual age for c++	IBM
visual age for cobol	IBM
db2	IBM
pentium II	CDMarket
pentium II	Compucompras
pentium II	Florida I
compaq presario	Florida I
ibm 3709 notebook	CDMarket
ibm 3709 notebook	Compucompras
ibm 3709 notebook	Compushow
ibm 3709 notebook	Florida I
ibm 3709 notebook	IBM

Notar que el articulo 'monitor cristal liquido' se perdió porque nadie lo proveía.

Respuesta 2.3)

```
SELECT descripcion, nombre
FROM articulo NATURAL JOIN provee
```

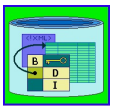
O bien

```
SELECT descripcion, nombre
FROM articulo JOIN provee ON articulo.codigo= provee.codigo
```

O bien

```
SELECT descripcion, nombre
FROM articulo, provee
WHERE articulo.codigo= provee.codigo
```

Silvia Alicia Gómez & Leticia Irene Gómez



Ejercicio 2.4)

Hallar las descripciones de los artículos y los nombres de los provedores que los proveen, pero conservar en el resultado aquellos artículos que no están provistos por ningún proveedor. En el ejemplo se debería obtener:

descripcion	Nombre
visual age for smalltalk	IBM
visual age for c++	IBM
visual age for cobol	IBM
db2	IBM
Pentium II	CDMarket
Pentium II	Compucompras
Pentium II	Florida I
Compaq presario	Florida I
ibm 3709 notebook	CDMarket
ibm 3709 notebook	Compucompras
ibm 3709 notebook	Compushow
ibm 3709 notebook	Florida I
ibm 3709 notebook	IBM
monitor de cristal liquido	null

Respuestas 2.4)

```
SELECT descripcion, nombre
FROM articulo LEFT OUTER JOIN
                  provee ON articulo.codigo= provee.codigo
```

O bien

```
SELECT descripcion, nombre
FROM provee RIGHT OUTER JOIN
                  articulo ON articulo.codigo= provee.codigo
```

Ejercicio 2.5)

Listar los nombres de los departamentos que no venden artículos, ordenados descendentemente. Hacerlo de dos formas distintas: a) con cláusula IN; b) con cláusula EXISTS.

En el ejemplo se debería obtener:

Descripcion
Ventas
Tesoreria
RRHH

Respuesta 2.5)

a) con cláusula IN

```
SELECT nombre
FROM departamento
WHERE nombre NOT IN (SELECT  vendido_por FROM articulo)
ORDER BY nombre DESC
```

Silvia Alicia Gómez & Leticia Irene Gómez

b) con cláusula EXISTS

```
SELECT nombre
FROM departamento
WHERE NOT EXISTS
      (SELECT * FROM articulo WHERE vendido_por = nombre)
ORDER BY nombre DESC
```

O bien

```
SELECT nombre
FROM departamento
WHERE NOT EXISTS
      (SELECT stock FROM articulo WHERE vendido_por = nombre)
ORDER BY nombre DESC
```

Como se puede observar, cuando se usa el EXISTS o NOT EXISTS lo que importa es si el conjunto obtenido es o no vacío. Si la expresión anidada es correcta, puede omitirse el atributo y colocarse *

Ejercicio 2.6)

Listar la descripción del producto más caro que no sea provisto por "Florida I". En el ejemplo se debería obtener:

descripcion
ibm 3709 notebook

Respuesta 2.6)

```
SELECT descripcion
FROM articulo, provee p1
WHERE articulo.codigo=p1.codigo AND nombre <> 'Florida I'
     AND NOT EXISTS(SELECT *
                    FROM provee p2
                    WHERE p2.nombre <>'Florida I' AND
                          p2.precio > p1.precio )
```

O bien

```
SELECT descripcion
FROM articulo NATURAL JOIN provee
WHERE  nombre <> 'Florida I'
       AND  precio =  (SELECT MAX(precio)
                       FROM provee
                        WHERE nombre <> 'Florida I')
```

O bien

```
SELECT descripcion
FROM articulo NATURAL JOIN provee
WHERE  nombre <> 'Florida I'
      AND  precio >=ALL (SELECT precio
                         FROM provee
                         WHERE nombre <> 'Florida I')
```

Silvia Alicia Gómez & Leticia Irene Gómez

Silvia Alicia Gómez & Leticia Irene Gómez

Ejercicio 2.7)

Encontrar los códigos de productos provistos por más de un proveedor. En el ejemplo se debería obtener:

Codigo
60
100

Respuesta 2.7)

La forma correcta es:

```
SELECT DISTINCT prov1.codigo
FROM provee prov1, provee prov2
WHERE prov1.codigo = prov2.codigo AND prov1.nombre <> prov2.nombre
```

O bien

```
SELECT DISTINCT prov1.codigo
FROM provee prov1
WHERE prov1.codigo IN
(SELECT codigo
FROM provee prov
WHERE prov1.nombre <> nombre)
```

O bien

```
SELECT DISTINCT prov1.codigo
FROM provee prov1
WHERE EXISTS
(SELECT *
FROM provee prov2
WHERE prov2.codigo=prov1.codigo AND prov2.nombre <> prov1.nombre)
```

Hacerlo así es incorrecto:

```
SELECT prov1.codigo
FROM provee prov1, provee prov2
WHERE prov1.codigo = prov2.codigo
          AND prov1.nombre <> prov2.nombre
```

porque se obtendría:

Codigo	
60	6 filas con condigo 60
....	
60	
100	20 filas con condigo 100
....	
100	

Ejercicio 2.8)

Listar el nombre del gerente. En el ejemplo se debería obtener:

Nombre
Diego

Respuesta 2.8)

```
SELECT nombre
FROM empleado
WHERE trabaja_en IS NULL
```

Ejercicio 2.9)

Listar los nombres de empleados que no son jefes ni gerente. Deberíamos obtener:

nombre
Gladys
Claudio
Pedro
Pablo
Juan
Clara

Respuesta 2.9)

```
SELECT nombre
FROM empleado
WHERE nombre NOT IN (SELECT jefe FROM DEPARTAMENTO)
     AND trabaja_en IS NOT null
```

O bien

```
SELECT nombre
FROM empleado
WHERE NOT EXISTS
          (SELECT * FROM departamento
            WHERE empleado.nombre = jefe)
                 AND trabaja_en IS NOT null
                 }
```

Ejercicio 2.10)

Listar el precio del producto más caro. En el ejemplo se debería obtener:

(max)
5200

Silvia Alicia Gómez & Leticia Irene Gómez

Respuesta 2.10)

```
SELECT precio
FROM provee AS p1
WHERE NOT EXISTS
(SELECT * FROM provee AS p2 WHERE p2.precio > p1.precio)
```

Ejercicio 2.11)

Listar el precio del producto más caro de hardware y el precio del producto más caro de software. (sin GROU BY). En el ejemplo se debería obtener:

(max)
1100
5200

Respuesta 2.11)

.

Como sabemos que solo hay dos tipos de productos: hardware y software, podemos resolverlo. Pero si hubiera más casos no puede armarse "unions" sin saber cuántos son y hay que recurrir a la cláusula GROUP BY

```
   (SELECT  max(precio)
   FROM articulo, provee
   WHERE articulo.codigo=provee.codigo AND vendido_por='Software')
UNION
   (SELECT  max(precio)
   FROM articulo, provee
   WHERE articulo.codigo=provee.codigo AND vendido_por='Hardware')
```

***Nota**:* Si justo ambos precios coincidieran se obtendría una sola tupla (la unión elimina repetidos). Si se quisiera mantener las repeticiones habría que hacer UNION ALL

Ejercicio 2.12)

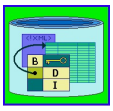
Listar las descripciones de los productos más baratos (podría haber más de uno). En el ejemplo debería obtenerse:

Descripción
visual age for cobol

Respuesta 2.12)

```
SELECT DISTINCT descripcion
FROM articulo, provee
WHERE articulo.codigo = provee.codigo
AND precio <= ALL  ( SELECT precio
                     FROM provee )
```

Silvia Alicia Gómez & Leticia Irene Gómez



Ejercicio 2.13)

Mostrar los nombres de los proveedores que abastecen a todos los productos que son de software (Llegó el operador %). Hacerlo de dos formas distintas: a) al estilo álgebra relacional; b) usando la función de agregación COUNT.

En nuestro ejemplo debería obtenerse:

nombre
IBM

Respuesta 2.13)

a) Al estilo Algebra Relacional o Cálculo Relacional (sin funciones de agregación)

Notar que el cálculo hubiéramos expresado

{ t | ∃p (Provee(p) ∧
∀a (Articulo(a) ∧ a[vendido_por]='Software' ⇒
∃h (Provee(h) ^ h[codigo]=a[codigo] ∧ p[name]=h[name])) ∧
t[nombre]=p[nombre]) }

Aunque sabemos que esa consulta no es correcta porque utiliza un símbolo no existente =>
Lo reescribimos recordando que ∀x (∃y (P(x, y)) equivale a ¬∃x (¬∃y (P(x, y)))

{ t | ∃p (Provee(p) ∧
¬∃a (Articulo(a) ∧ a[vendido_por]='Software' ∧
¬∃h (Provee(h) ^ a[codigo]=h[codigo] ∧ p[name]=h[name])) ∧
t[nombre]=p[nombre]) }

Ahora escribiéndolo en SQL obtenemos, que como se observa en el coloreado, es la traducción exacta de la expresión del Cálculo Relacional al SQL:

```
SELECT DISTINCT p.nombre
FROM provee p
WHERE NOT EXISTS (SELECT *
                  FROM articulo a
                  WHERE a.vendido_por = 'Software' AND
                        NOT EXISTS
                        (SELECT *
                        FROM provee h
                        WHERE a.codigo = h.codigo AND
                              p.nombre = h.nombre) )
```

O bien esta otra solución, que es mucho mejor, pues fija "nombre" de la tabla PROVEEDOR que tiene menos tuplas y no hay que usar DISTINCT:

Silvia Alicia Gómez & Leticia Irene Gómez

```
SELECT p.nombre
FROM proveedor p
WHERE NOT EXISTS (SELECT *
                  FROM articulo a
                  WHERE a.vendido_por = 'Software'
                        AND NOT EXISTS
                           (SELECT *
                            FROM provee h
                            WHERE a.codigo = h.codigo
                            AND p.nombre = h.nombre) )
```

Silvia Alicia Gómez & Leticia Irene Gómez